

TIA 4: Guía de Laboratorio No. 7 Interfaz

1) Interfaz vs Clase Concreta

Las interfaces son más flexibles porque definen un contrato, no una implementación específica. Así puedes cambiar fácilmente las clases que las implementan. Ejemplo: cualquier pizza que cumpla la receta sirve.

2) Separar lógica de entrega en Pedido

Para seguir el principio SOLID de "una clase = una responsabilidad". Pedido sólo maneja datos del pedido, no cómo se entrega. Más fácil de mantener.

3) Principio SOLID más importante aquí

Inversión de Dependencias (D): Pedido depende de la interfaz IMetodoEntrega, no de clases concretas. Así se pueden cambiar métodos de entrega fácilmente.

4) Añadir entrega en bicicleta

- Crear clase EntregaBicicleta (implementa IMetodoEntrega).
- Modificar EntregaFactory para incluirla, no se toca el pedido ni la interfaz!

5) Ventajas de estos patrones

Strategy: Cambia cómo se entrega sin modificar Pedido.

Factory: Toda la creación de entregas en un solo lugar.

Singleton: Un único registro de pedidos.

6) ¿Cuándo usar Singleton?

Cuando se necesita una sola instancia global, como: Registro de usuarios conectados, configuraciones de la app, Log de errores.